

Building a MySQL-based RAG System from Scratch

VECTOR Type in Practice, Engineering Trade-offs, and a pgvector Comparison

Hank (Green Bean Soup / litotom)

COSCUP 2026 · MySQL Track



Slides and sample code · <https://github.com/hanktom/coscup-mysql>

About me

- 30 years in software: programmer → software manager → technical consultant → director
- Founder of the **Green Bean Soup** tech blog (litotom)
- The 18th Google Certified Android Developer in the world
- Today's role: **someone who speaks for the DBA**

LitoShop's support crisis

Fictional store **LitoShop**. Support gets asked this every day:

“How long until my refund arrives?”

“When will the money go back to my card?”

“I was refunded — why isn't it on my statement yet?”

Same answer. A hundred ways to ask it.

Leadership says: ship AI support.

Then came that meeting

Architect: “To do RAG, we have to add a vector database. Pinecone? Qdrant? Or pgvector?”

DBA: “...don’t we already have MySQL?”

(The room goes quiet for three seconds.)

This talk is the full technical answer after those three seconds.

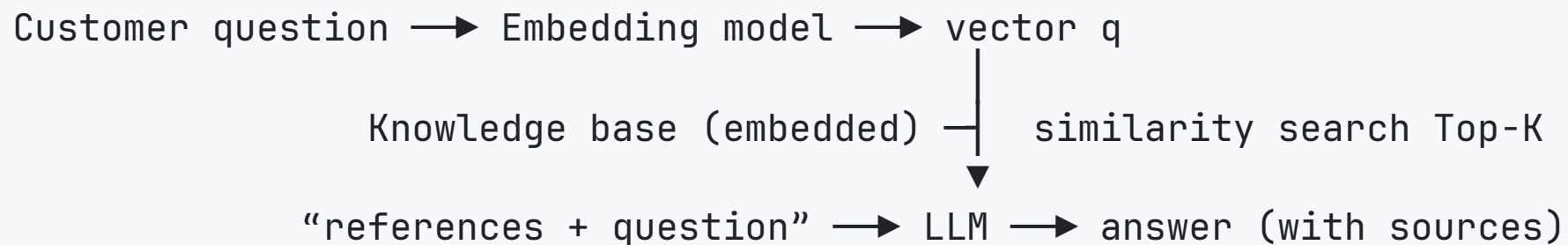
The question we will answer today

“We only have MySQL. Can we do RAG without standing up another database?”

- What MySQL 9.x `VECTOR` can and cannot do
- A from-scratch AI-support demo (fully open source, reproducible)
- Same data, same queries, an honest pgvector comparison
- A decision framework you can take home

Out of scope: LLM training, vector math, HeatWave and other cloud-only features

RAG on one slide



- Embedding: turn meaning into a vector (today: **bge-m3**, 1024 dims, MIT license)
- Retrieval: **find the nearest K vectors in a pile — that is the database’s job**
- Today has one star: **the retrieval-layer database**

How RAG storage gets discussed today

The storage layers people reach for:

Pinecone | Qdrant | Milvus | Weaviate | **PostgreSQL + pgvector**

Nobody mentions MySQL.

But the reality is—

MySQL is the most common OLTP database already in the enterprise stack.

Your orders, members, and products already live there.

The real reasons to “just use MySQL”

- **Don't move the data:** knowledge base and transactions in one database; `JOIN` them directly
- **Don't add ops surface:** no new database to back up, monitor, upgrade, or staff
- **Permissions and audit:** reuse the existing accounts and audit process
- **Team skills:** the DBA is already here

That is not laziness. It is an **honest engineering-cost calculation**.

Only one question left: can MySQL actually do it?

The VECTOR type in MySQL 9.x

MySQL 9.0 (2024-07) officially introduced:

```
CREATE TABLE faq_chunks (  
  id          INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
  category    VARCHAR(32) NOT NULL,  
  content     TEXT        NOT NULL,  
  embedding   VECTOR(1024) NOT NULL      -- ◀ the star  
);
```

- 4 bytes per dim (float32) binary storage, max 16,383 dims
- 1024 dims $\approx$ 4 KB / row; 100,000 rows $\approx$ 400 MB
- InnoDB, binlog, replication all work as usual — **it is just a column**

Community Edition gives you three functions. That's it.

```
SELECT STRING_TO_VECTOR('[0.1, 0.2, 0.3]'); -- write
SELECT VECTOR_TO_STRING(embedding);          -- read as text
SELECT VECTOR_DIM(embedding);                 -- dimension
```

So... where is the similarity function?

Live: let's compute a distance

```
mysql> SELECT DISTANCE(  
→   embedding,  
→   STRING_TO_VECTOR('[0.1, 0.2, ...]'),  
→   'COSINE'  
→ ) FROM faq_chunks LIMIT 1;
```

```
ERROR 1305 (42000): FUNCTION ragdemo.DISTANCE does not exist
```

Official docs: *"DISTANCE() ... available only for MySQL HeatWave on OCI"*

Current state (through MySQL 9.7, 2026-04 GA)

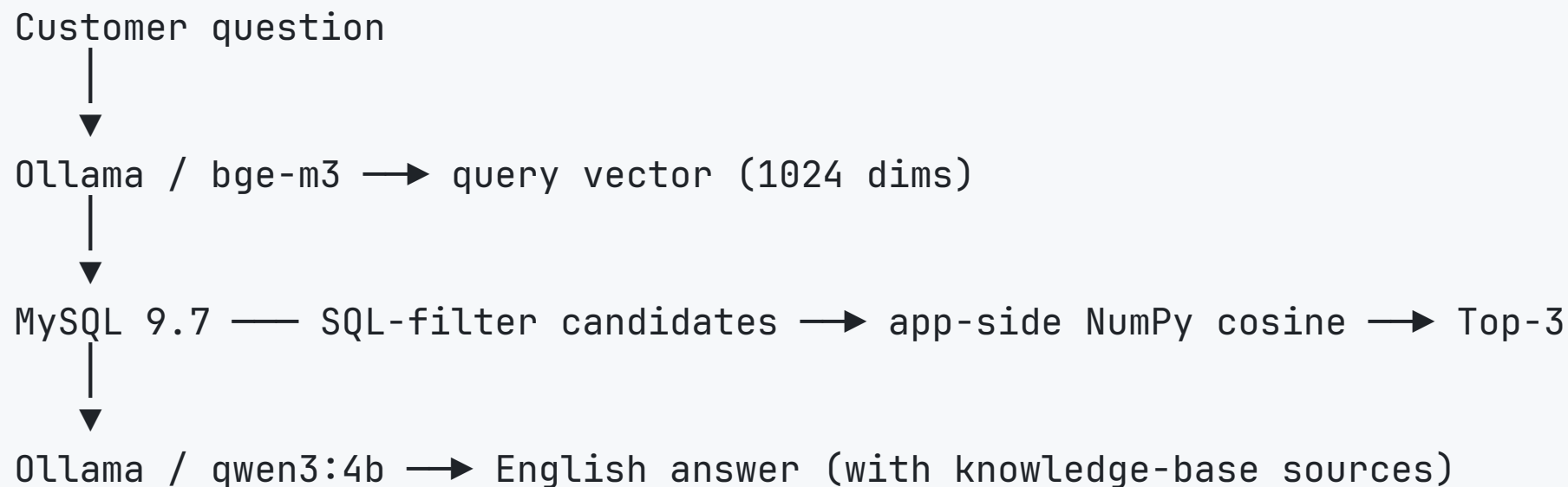
Capability	Community	HeatWave (OCI)
<code>VECTOR</code> type storage	Yes	Yes
Conversion helpers (<code>STRING_TO_VECTOR</code> , ...)	Yes	Yes
<code>DISTANCE()</code> function	No	Yes
ANN indexes (HNSW / IVF)	No	Yes

You got the type, not the engine.

“A car with seats and a steering wheel — but no engine.”

So we install the engine ourselves: compute distance in the application.

Demo architecture: LitoShop AI support



Fully open source, fully local, reproducible with `docker compose up`

Knowledge base: synthetic LitoShop support FAQ (CC0, 43 chunks / 10 categories)

Retrieval strategy: MySQL stores, Python is the engine

```
cur.execute("SELECT id, title, content, embedding FROM faq_chunks")
rows = cur.fetchall()

# VECTOR comes back as float32 binary → frombuffer directly, zero parse cost
mat = np.frombuffer(b"".join(r[3] for r in rows), dtype=np.float32)
mat = mat.reshape(len(rows), 1024)

scores = mat @ query_vec          # already normalized: dot product = cosine
topk    = np.argsort(-scores)[:3]
```

Two details that matter:

- Read the **raw binary**. Do not use `VECTOR_TO_STRING` (skip a text parse)
- Normalize embeddings first; cosine collapses to one matrix multiply

▶ Live Demo I

LitoShop AI support on MySQL

“How long until my refund arrives?” → “When will the money go back to my card?”

Is there a fee to exchange sizes? | Is an ATM “cancel installment” call a scam?

Demo I recap

- 43 knowledge chunks, a few hundred FAQ rows: retrieval in **2–3 ms**
- Completely different wording still hits — keyword search cannot do this
- RAG makes the LLM say the *right* thing: answers have sources and can be audited

At this scale, MySQL is enough.

Can we adjourn the meeting?

Not yet: the knowledge base will grow

A support FAQ is only the start. Next in the door:

- Site-wide **product copy** (tens of thousands of rows)
- Historical **support tickets** (hundreds of thousands of rows)
- Articles, spec sheets, reviews...

At 100,000 vectors, does the setup we just used still hold?

Benchmark: same data, same queries

(Measured in a 7 GB RAM container, p50; on stage, rerun on the speaker's VM)

Scenario	10k rows	100k rows
MySQL full scan + app-side sort	242 ms	2,144 ms
MySQL category filter (cut to 1/10)	38 ms	244 ms
pgvector no index (exact)	39 ms	469 ms
pgvector HNSW	2.2 ms	9.7 ms

Where do the 2 seconds come from?

On every query, the MySQL approach does this:

Move 400 MB of vectors out of the database, then compute over all of them.

- The bottleneck is not NumPy (the matrix multiply is ~5%)
- The bottleneck is **network transfer + driver deserialization**
- 10× the data → 10× the latency — **linear, no surprises**

Why is pgvector fast? Distance stays **inside the database**, and HNSW means you **do not compute everything**.

Workarounds from the MySQL camp

- **Metadata coarse filter** (most effective): support naturally has `category` / `tenant`
→ one `WHERE` drops 90% of candidates, 2,144 ms → 244 ms
- **Partition by tenant**: multi-tenant SaaS; each tenant stays a small full scan
- **Covering index**: `(category, id)` + two-step vector fetch, less data movement
- **Read replica**: offload vector queries; do not fight OLTP for the buffer pool

Honestly: these delay the collapse. They are not a substitute for ANN.

Control group: PostgreSQL + pgvector

- Open-source extension (PostgreSQL License), currently 0.8.x
- `vector` type + **6 distance operators** + **HNSW / IVFFlat indexes**
- 0.8 iterative index scan: filters and ANN indexes cooperate more smartly

```
SELECT doc_id, title, embedding <=> :query_vec AS dist
FROM   faq_chunks
ORDER BY embedding <=> :query_vec      -- distance computed in the DB
LIMIT 5;                             -- with HNSW this is ANN
```

Same chatbot. One flag: `--db pg`.

► Live Demo II

Same data, moved into pgvector

```
ORDER BY embedding <=> q | EXPLAIN ANALYZE
```

HNSW index: before vs after

EXPLAIN before and after (100k rows)

Before the index:

```
Limit ... Sort ... Seq Scan on faq_chunks  
Execution Time: ~470 ms
```

After `CREATE INDEX ... USING hnsw (embedding vector_cosine_ops) :`

```
Limit ... Index Scan using idx_faq_embedding on faq_chunks  
Execution Time: ~10 ms
```

One DDL statement, a 47× difference. **That is the value of an ANN index.**

One-page comparison

	MySQL 9.7 Community	PostgreSQL + pgvector 0.8
Vector type	<code>VECTOR</code> ($\leq 16,383$ dims)	<code>vector</code> / <code>halfvec</code> / <code>sparsevec</code>
Distance	None (app-side)	6 operators, inside the DB
ANN index	None	HNSW / IVFFlat
Filter + vector search	SQL filter $\rightarrow$ app sort	iterative index scan
Extra ops cost	Zero (existing stack)	Another PG (if you do not have one)
JOIN to transactional data	Same database, direct JOIN	Depends on where the data lives

pgvector is not a free lunch either

- **HNSW build:** 100k rows × 1024 dims takes on the order of minutes + enough `maintenance_work_mem`
(we already hit the too-small shm pit in the demo environment)
- **ANN = approximate:** recall is not 100%; tune `ef_search`
- **Another database:** backup, HA, upgrades, monitoring, people — every reason you did not want to move is still here
- Data has to be **synced** over: CDC or dual-write; consistency is on you

Choosing pgvector is not “flip a switch.” It is “take on another database.”

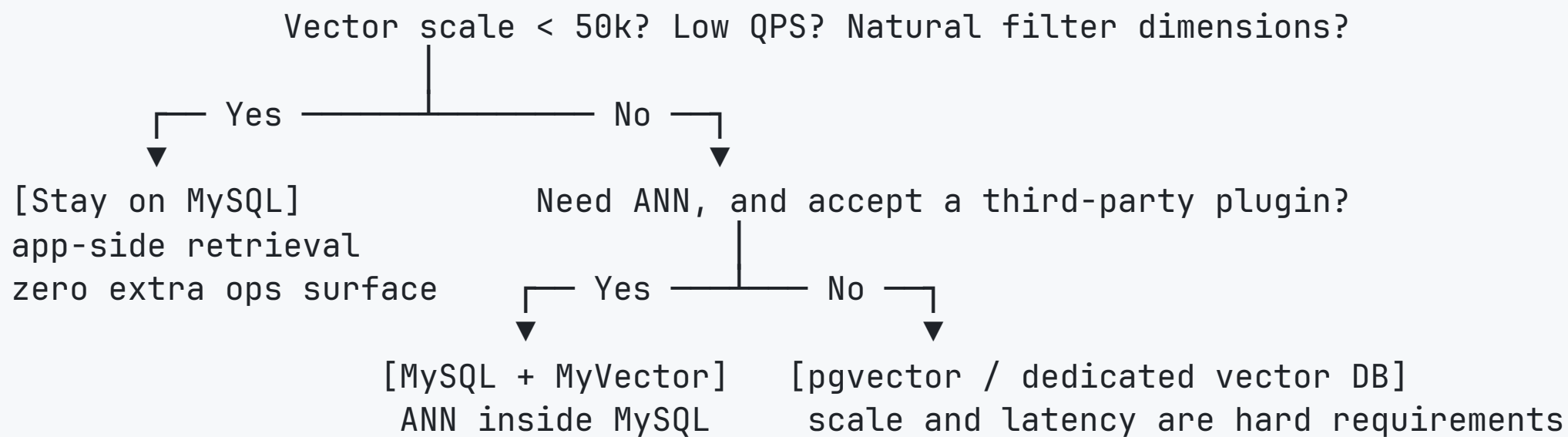
Third path: the community installs the engine — MyVector

Open-source (GPLv2) MySQL plugin / component. HNSW ANN lands inside MySQL:

```
CREATE TABLE faq (  
  id INT PRIMARY KEY,  
  vec VARBINARY(4096) COMMENT 'MYVECTOR(type=HNSW,dim=1024,dist=COSINE)'  
);  
CALL mysql.myvector_index_build('ragdemo.faq.vec', 'id');  
  
SELECT id, myvector_row_distance() AS d FROM faq  
WHERE MYVECTOR_IS_ANN('ragdemo.faq.vec', 'id', @qvec, 10);
```

- Index synced via binlog; supports MySQL 8.0 / 8.4 / 9.7
- Positioning: for people who want to stay on MySQL and really need ANN
- Judge the risk yourself: third-party plugin upgrade cycle, support capacity, community size

Three-way decision framework



Decision factors are not only latency

Factor	Tips toward MySQL	Tips toward pgvector / a vector DB
Vector scale	< 50k	> 100k and still growing
Query latency	Seconds OK / offline	Online p95 < 100 ms
Filter dimensions	Natural multi-tenant / category	Global, cross-boundary search
Team today	MySQL DBAs only	Already run Postgres
Data gravity	Must JOIN transactional data	Knowledge base is independent

There is no correct answer. There is only your situation.

Three things to take home

1. A demo that actually runs

`docker compose up` → LitoShop AI support (MySQL + pgvector dual backends)

2. An honest comparison table

Including the benchmark script — rerun it at home on your own data

3. A decision framework

Next time the room goes quiet for three seconds, you have an answer

Community for the common good

- This repo: code **Apache-2.0**, content **CC BY-SA 4.0**, dataset **CC0**
- MySQL vector search is still on the roadmap —
Vote, comment, and write on official channels so Oracle hears what the community wants
- MariaDB 11.7 native VECTOR indexes, MyVector — the ecosystem is moving; the story is not over

Thank you!

Q&A

repo: [github.com/<your-repo>](https://github.com/litotom/green-bean-soup) (QR code)

Green Bean Soup / litotom

Code Apache-2.0 · Slides CC BY-SA 4.0 · Dataset CC0